

PATRÓN PESO MOSCA (FLYWEIGHT)

Capítulo 17 — Patrones de Diseño en Java

Traducción técnica al español

1. Descripción

Todo objeto puede concebirse como compuesto por uno o ambos de los siguientes conjuntos de información:

1. Información intrínseca: la información intrínseca de un objeto es independiente del contexto del objeto. Esto significa que la información intrínseca es la información común que permanece constante entre diferentes instancias de una clase determinada. Por ejemplo, la información de la empresa en una tarjeta de visita es la misma para todos los empleados.
2. Información extrínseca: la información extrínseca de un objeto depende del contexto del objeto y varía según él. Esto significa que la información extrínseca es única para cada instancia de una clase determinada. Por ejemplo, el nombre y el cargo del empleado son extrínsecos en una tarjeta de visita, ya que esta información es diferente para cada empleado.

Considérese un escenario de aplicación que implique la creación de una gran cantidad de objetos que son únicos solo en términos de unos pocos parámetros. En otras palabras, estos objetos contienen cierta información intrínseca e invariante que es común entre ellos. Esta información intrínseca debe ser creada y mantenida como parte de cada objeto que se crea. La creación y el mantenimiento generales de una gran cantidad de dichos objetos puede ser muy costoso en términos de uso de memoria y tiempo.

El patrón Peso Mosca puede utilizarse en tales escenarios para diseñar una forma más eficiente de crear objetos. El patrón sugiere separar toda la información intrínseca común en un objeto separado denominado objeto Flyweight (Peso Mosca). El grupo de objetos creados puede compartir el objeto Flyweight ya que representa su información intrínseca. Esto elimina la necesidad de almacenar la misma información invariante e intrínseca en cada objeto; en cambio, se almacena solo una vez en forma de un único objeto Flyweight. Como resultado, la aplicación cliente puede lograr ahorros considerables en términos de uso de memoria y tiempo.

2. Requisitos del Patrón Flyweight

Cuando se aplica el patrón Flyweight, es importante asegurarse de que se cumplan los siguientes requisitos:

Nº	Descripción
1	Solo existe un objeto de un tipo flyweight determinado y este es compartido por todos los otros objetos apropiados.
2	Los objetos cliente no deben tener permitido crear instancias flyweight directamente. Al mismo tiempo, los objetos cliente deben tener una manera de obtener acceso a un objeto Flyweight requerido cuando sea necesario.

Tabla 17.1 — Requisitos del patrón Flyweight

3. Cómo Diseñar un Flyweight en Java

Una de las formas de diseñar un flyweight en Java es diseñarlo como un singleton. El objeto Flyweight se diseña con un constructor privado para impedir que los objetos cliente creen instancias de Flyweight accediendo directamente a su constructor.

En general, un singleton mantiene solo una instancia a nivel del tipo de clase. Cuando un flyweight se diseña como singleton, exhibe la naturaleza singleton a nivel del tipo flyweight, no a nivel del tipo de clase. En otras palabras, el Flyweight mantiene una única instancia de sí mismo para cada tipo de flyweight posible en el sistema. Estos objetos flyweight se almacenan en la variable estática `IstFlyweight` (como un `HashMap`). Esto puede verse como una variación del patrón Singleton.

Siempre que un cliente necesita crear una instancia de un flyweight determinado, invoca el método estático `getFlyweight`, pasando el tipo de flyweight requerido como argumento. El método `getFlyweight` está diseñado como un método sincronizado a nivel de clase para hacerlo seguro para hilos (thread-safe).

Como parte de su implementación del método `getFlyweight`, el Flyweight verifica si ya existe en el `HashMap IstFlyweight` una instancia correspondiente al tipo de flyweight solicitado:

- Si existe, el Flyweight retorna el objeto Flyweight existente al cliente.
- Si no existe: el Flyweight crea una nueva instancia de sí mismo correspondiente al tipo de flyweight solicitado y la agrega a la lista de flyweights mantenida en la variable estática `IstFlyweight`. Dado que el método `getFlyweight` está definido dentro de la clase Flyweight, puede acceder a su constructor privado para crear una instancia de sí mismo. Luego retorna el objeto Flyweight recién creado al cliente.

3.1 Estrategia Alternativa: FlyweightFactory

Como estrategia de diseño alternativa, la responsabilidad de crear y mantener los diferentes objetos Flyweight singleton puede trasladarse fuera de la clase Flyweight a una clase designada FlyweightFactory. El Flyweight puede diseñarse como una clase interna (inner class) de la clase FlyweightFactory. Dado que la clase Flyweight se diseña con un constructor privado, los objetos externos no pueden crear instancias invocando directamente el constructor. Sin embargo, la clase FlyweightFactory puede acceder al constructor privado de la clase Flyweight para crear los

objetos Flyweight necesarios, ya que una clase externa puede acceder a los métodos privados de su clase interna.

En el nuevo diseño, tanto la estructura de datos (IstFlyweight HashMap) como el comportamiento (método getFlyweight) relacionados con la creación y el mantenimiento de los objetos Flyweight singleton se trasladan de la clase Flyweight a la clase FlyweightFactory. La clase Flyweight se utiliza únicamente para representar el estado intrínseco de un objeto. La FlyweightFactory se diseña como singleton para impedir que los clientes creen múltiples instancias de ella, lo que podría derivar en múltiples instancias de un tipo de flyweight determinado.

4. Ejemplo — Tarjetas de Visita

Para demostrar el uso del patrón Flyweight, se diseña una aplicación que imprime los datos de las tarjetas de visita de todos los empleados de una gran organización con cuatro oficinas divisionales principales. El diseño típico de una tarjeta de visita puede asumir la siguiente estructura:

<Nombre del Empleado> <Cargo> <Nombre de la Empresa> <Líneas de Dirección de la Oficina Divisional> <Ciudad>, <Estado> <Código Postal>

A partir del diseño de los datos de la tarjeta de visita, se puede observar que:

- El nombre y el cargo son únicos para cada empleado y se consideran como datos extrínsecos.
- El nombre de la empresa permanece igual para todos los empleados, y cada empleado que trabaja en una oficina divisional recibe la misma dirección de esa oficina. Por lo tanto, el nombre de la empresa y la dirección divisional en una tarjeta de visita pueden tratarse como datos intrínsecos.

Normalmente habrá miles de empleados en una organización grande, por lo que la aplicación puede necesitar crear miles de objetos VCard. Como se discutió anteriormente, la parte de la dirección de la clase VCard permanece constante para los empleados que trabajan en una misma oficina divisional. Por lo tanto, sin el patrón aplicado, podría resultar en datos duplicados siendo creados y mantenidos como parte de cada nueva instancia de VCard.

4.1 Enfoque de Diseño I — Datos Extrínsecos como Objeto

En este enfoque, los datos extrínsecos se representan como un objeto y se asocian con un objeto Flyweight que representa sus datos intrínsecos. Aplicando el patrón Flyweight, todos los datos intrínsecos se mueven de la clase VCard a una clase Flyweight separada.

Se define una interfaz FlyweightIntr a ser implementada por la clase Flyweight que representa los datos intrínsecos de la tarjeta de visita:

```
public interface FlyweightIntr {
```

```
    public String getCompany();
    public String getAddress();
    public String getCity();
    public String getState();
    public String getZip();
}
```

Se define una clase `FlyweightFactory` singleton (Listado 17.1) con la responsabilidad de crear y mantener instancias únicas de diferentes objetos `Flyweight` correspondientes a diferentes divisiones. La clase `Flyweight` concreta se define dentro de la `FlyweightFactory` como clase interna:

```
// Singleton FlyweightFactory
public class FlyweightFactory {
    private HashMap lstFlyweight;
    private static FlyweightFactory factory =
        new FlyweightFactory();

    private FlyweightFactory() {
        lstFlyweight = new HashMap();
    }

    public synchronized FlyweightIntr getFlyweight(
        String divisionName) {
        if (lstFlyweight.get(divisionName) == null) {
            FlyweightIntr fw = new Flyweight(divisionName);
            lstFlyweight.put(divisionName, fw);
            return fw;
        } else {
            return (FlyweightIntr) lstFlyweight.get(divisionName);
        }
    }

    public static FlyweightFactory getInstance() {
        return factory;
    }

    // Clase interna Flyweight
    private class Flyweight implements FlyweightIntr {
        private String company;
        private String address;
        private String city;
        private String state;
        private String zip;

        private Flyweight(String division) {
            // Valores asignados según la división
        }
    }
}
```

```

        if (division.equals("North")) setValues(...);
        if (division.equals("South")) setValues(...);
        if (division.equals("East"))  setValues(...);
        if (division.equals("West"))  setValues(...);
    }
    public String getCompany() { return company; }
    public String getAddress() { return address; }
    public String getCity()    { return city; }
    public String getState()   { return state; }
    public String getZip()     { return zip; }
} // fin de Flyweight
} // fin de FlyweightFactory

```

Tras extraer los datos intrínsecos, los datos extrínsecos permanecen en la clase VCard. Como parte de su constructor, el VCard recibe una instancia del Flyweight que representa sus datos intrínsecos:

```

public class VCard {
    String name;
    String title;
    FlyweightIntr objFW;

    public VCard(String n, String t, FlyweightIntr fw) {
        name = n;
        title = t;
        objFW = fw;
    }

    public void print() {
        System.out.println(name);
        System.out.println(title);
        System.out.println(objFW.getAddress() + ", " +
            objFW.getCity() + ", " +
            objFW.getState() + " " + objFW.getZip());
    }
}

```

El cliente FlyweightTest para imprimir los datos de la tarjeta de visita:

3. Crea una instancia del singleton FlyweightFactory.
4. Para cada empleado, solicita al FlyweightFactory el objeto Flyweight apropiado invocando el método getFlyweight y pasando la división en la que trabaja el empleado. En respuesta, la FlyweightFactory retorna el objeto Flyweight correspondiente a la división especificada. Dado que la organización tiene solo cuatro divisiones, se crearán como máximo cuatro objetos Flyweight cuando se ejecute la aplicación.
5. Recibe el objeto Flyweight solicitado y luego asocia el flyweight con la instancia VCard que representa los datos extrínsecos, pasando el objeto Flyweight como argumento al constructor del VCard.

Este enfoque requiere la creación de un objeto VCard por cada empleado, además de los cuatro objetos Flyweight. El hecho de que los datos intrínsecos no se dupliquen en cada objeto VCard genera ahorros en términos del uso de memoria y el tiempo que se tarda en crear objetos.

4.2 Enfoque de Diseño II — Datos Extrínsecos Pasados como Parámetro

En este enfoque, los datos extrínsecos se pasan al objeto Flyweight como parte de una llamada a un método, en lugar de ser representados como un objeto separado. Este enfoque de diseño requiere los siguientes dos cambios respecto al Enfoque I:

- El método print debe moverse de la clase VCard al objeto Flyweight, y su firma debe cambiar de `public void print()` a `public void print(String name, String title)`, para aceptar los datos extrínsecos como argumentos. Debe implementarse para mostrar los datos extrínsecos recibidos junto con los datos intrínsecos que representa.
- La clase VCard: dado que los datos extrínsecos ahora se pasan al objeto Flyweight como parte de una llamada a método, la clase VCard ya no es necesaria y puede eliminarse del diseño.

La interfaz FlyweightIntr revisada incluye el nuevo método print:

```
public interface FlyweightIntr {
    public String getCompany();
    public String getAddress();
    public String getCity();
    public String getState();
    public String getZip();
    public void print(String name, String title);
}
```

En el nuevo diseño, el cliente FlyweightTest:

- Primero crea una instancia del singleton FlyweightFactory (la estructura e implementación de la FlyweightFactory permanece igual que en el Enfoque I).
- Para cada empleado, solicita a la FlyweightFactory el objeto Flyweight apropiado pasando la división en la que trabaja el empleado.
- Una vez recibido el objeto Flyweight solicitado, el cliente invoca el método print sobre el objeto Flyweight pasando los datos extrínsecos del empleado (nombre y cargo).

```
// Fragmento del cliente FlyweightTest (Enfoque II)
FlyweightFactory factory = FlyweightFactory.getInstance();
for (int i = 0; i < empList.size(); i++) {
    // ... parsear nombre, titulo, division ...
    FlyweightIntr flyweight = factory.getFlyweight(division);
    // Pasar datos extrínsecos como parte de la llamada al método
    flyweight.print(name, title);
}
```

}

Dado que los datos extrínsecos no se diseñan como un objeto, este enfoque requiere la creación de solo cuatro objetos Flyweight, uno por cada oficina divisional, sin duplicación alguna. Esto reduce sustancialmente el uso de memoria y el tiempo requerido para la creación de objetos.

4.3 Comparación de Ambos Enfoques

Aspecto	Enfoque I	Enfoque II
Datos extrínsecos	Representados como objeto VCard asociado al Flyweight.	Pasados como parámetros en la llamada al método print.
Objetos creados	Un objeto VCard por empleado + 4 objetos Flyweight.	Solo 4 objetos Flyweight (uno por división).
Clase VCard	Necesaria.	Eliminada del diseño.
Uso de memoria	Reducido (sin duplicar datos intrínsecos en cada VCard).	Mínimo (sin objetos VCard ni datos duplicados).
Método print	Reside en la clase VCard.	Reside en el objeto Flyweight (acepta datos extrínsecos).

Tabla 17.2 — Comparación entre Enfoque I y Enfoque II

5. Beneficios Clave del Diseño

- Reducción del uso de memoria: al compartir la información intrínseca entre múltiples objetos en lugar de duplicarla en cada uno, se logra un ahorro significativo de memoria, especialmente cuando se crean miles de objetos.
- Rendimiento mejorado: la reducción en el número de objetos creados disminuye también el tiempo de creación y la presión sobre el recolector de basura (garbage collector).
- Singleton a nivel de tipo flyweight: a diferencia del patrón Singleton clásico (una instancia por clase), el Flyweight garantiza una única instancia por tipo de flyweight, lo que lo convierte en una variación más granular del Singleton.
- Seguridad en entornos multihilo: el método getFlyweight está declarado como synchronized, garantizando que no se creen instancias duplicadas de un mismo tipo de flyweight en escenarios de concurrencia.
- Separación clara de responsabilidades: la FlyweightFactory gestiona el ciclo de vida de los objetos flyweight, mientras que el Flyweight solo representa el estado intrínseco, respetando el Principio de Responsabilidad Única.
- Clase interna como mecanismo de encapsulación: diseñar el Flyweight como clase interna de la FlyweightFactory permite que la fábrica acceda al constructor privado del flyweight sin exponerlo al exterior.

6. Preguntas de Práctica

1. Considérese un sitio de empleo en línea que recibe archivos XML de empleadores con las vacantes actuales en sus organizaciones. Cuando el número de vacantes es pequeño, los empleadores pueden ingresar los detalles en línea. Cuando el número de vacantes es grande, los empleadores cargan los detalles en forma de un archivo XML. Una vez recibido el archivo XML, debe ser analizado sintácticamente (parseado) y procesado. Se asume que el archivo XML contiene los siguientes detalles:

Campo	Tipo de dato
a. Cargo (Job title)	Extrínseco (varía por vacante)
b. Calificaciones mínimas	Extrínseco (varía por vacante)
c. Seguro médico	Intrínseco (común para todas las vacantes del empleador)
d. Seguro dental	Intrínseco
e. Atención visual	Intrínseco
f. Plan 401K	Intrínseco
g. Mínimo de horas de trabajo	Intrínseco
h. Vacaciones pagadas	Intrínseco
i. Nombre del empleador	Intrínseco
j. Dirección del empleador	Intrínseco

Los detalles (c) al (j) se consideran datos intrínsecos comunes para todas las vacantes publicadas por un empleador determinado. Aplicar el patrón Flyweight para diseñar el proceso de análisis del archivo XML de entrada y la creación de objetos Job.

2. En una organización típica, un usuario está asociado a una cuenta de usuario que puede pertenecer a uno o más grupos. Los permisos sobre diferentes recursos se definen a nivel de grupo. Se considera una organización con tres grupos: Administrators, FieldOfficers y SalesReps. La organización está en proceso de migrar cuentas de usuario a un nuevo entorno de servidor. Como parte de este proceso, las cuentas se exportan primero a un archivo XML. Se debe notar que los permisos para todas las cuentas de un grupo determinado son los mismos (dato intrínseco). El nombre de usuario y la contraseña varían de usuario a usuario y deben tratarse como datos extrínsecos. Realizar las suposiciones necesarias y diseñar una aplicación usando el patrón Flyweight para analizar el archivo XML y crear los objetos de cuentas de usuario.